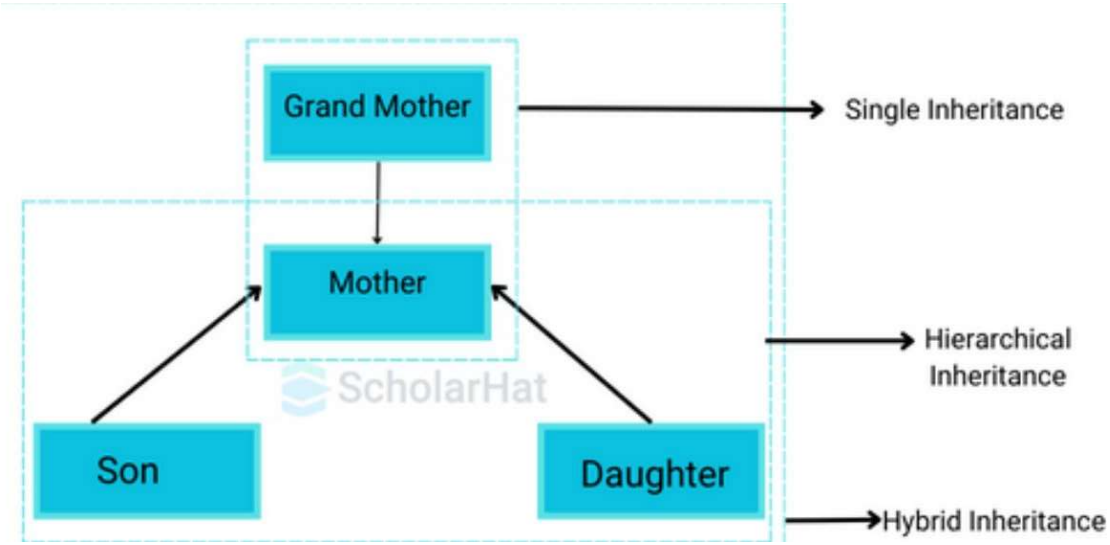
What is Inheritance?

Inheritance allows a **new class (child/subclass)** to acquire properties and methods of an **existing class (parent/superclass).**

Promotes:

- Code Reusability
- Extensibility
- Hierarchical classification
- Polymorphism

```
class Parent:
    pass

class Child(Parent):
    pass
```



Basic Inheritance

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hello, I am", self.name)


class Student(Person):
    pass


s = Student("Ram")
s.greet()
```

```
Hello, I am Ram
```

Customizing an Existing Class

We inherit and then:

- Add new methods
- Override existing methods
- Extend behavior

1. Method Overriding

```
class Person:
    def greet(self):
        print("Hello from Person")


class Student(Person):
    def greet(self):
        print("Hello from Student")


s = Student()
s.greet()
```

Hello from Student

Child method overrides parent method.

Using super() to Extend Behavior

```
class Person:
    def greet(self):
        print("Hello from Person")


class Student(Person):
    def greet(self):
        super().greet()
        print("Hello from Student")


s = Student()
s.greet()
```

```
Hello from Person
Hello from Student
```

This is customization without losing original behavior.

What is a Superclass?

A **Superclass (Parent Class)** is a class whose properties and methods are inherited by another class.

Child class = Subclass
Parent class = Superclass

```
class Animal:  # Superclass
    def speak(self):
        print("Animal speaks")

class Dog(Animal):  # Subclass
    pass

d = Dog()
d.speak()
```

Animal speaks

✔ Dog inherited speak()
✔ Animal is the superclass

What is a Superclass?

super() is used to:

- Call methods of the superclass
- Access parent class constructor
- Extend functionality instead of replacing it

super() in Method Overriding

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

d = Dog()
d.speak()
```

```
Bark
```

What is a Superclass?

super() in Method Overriding

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        print("Bark")

d = Dog()
d.speak()
```

```
Bark
```

With super()

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def speak(self):
        super().speak()
        print("Bark")

d = Dog()
d.speak()
```

```
Animal sound
Bark
```

✔ Parent behavior preserved
✔ Child behavior added

This is called **method extension.**

super() in Constructors (__init__)

- If parent has constructor, child must call it.

```
class Person:
    def __init__(self, name):
        self.name = name
        print("Person constructor")

class Student(Person):
    def __init__(self, name, roll):
        super().__init__(name)
        self.roll = roll
        print("Student constructor")

s = Student("Ram", 101)
```

✔ Parent constructor runs first
✔ Child constructor runs next

```
Person constructor
Student constructor
```

What Happens If You Don't Use super()?

```
class Student(Person):
    def __init__(self, name, roll):
        self.roll = roll

s = Student("Ram", 101)
print(s.name)
```

ERROR → name not defined

super() in Multiple Inheritance

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        super().show()
        print("B")

class C(B):
    def show(self):
        super().show()
        print("C")

obj = C()
obj.show()
```

```
A
B
C
```

Real-World Example

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        self.balance -= amount
        print("Withdrawn:", amount)


class SavingsAccount(BankAccount):
    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance")
        else:
            super().withdraw(amount)
```

Types of Inheritance in Python

1. Single Inheritance

One parent → One child

2.Multiple Inheritance

One child → Multiple parents

3. Multilevel Inheritance

Grandparent → Parent → Child

4. Hierarchical Inheritance

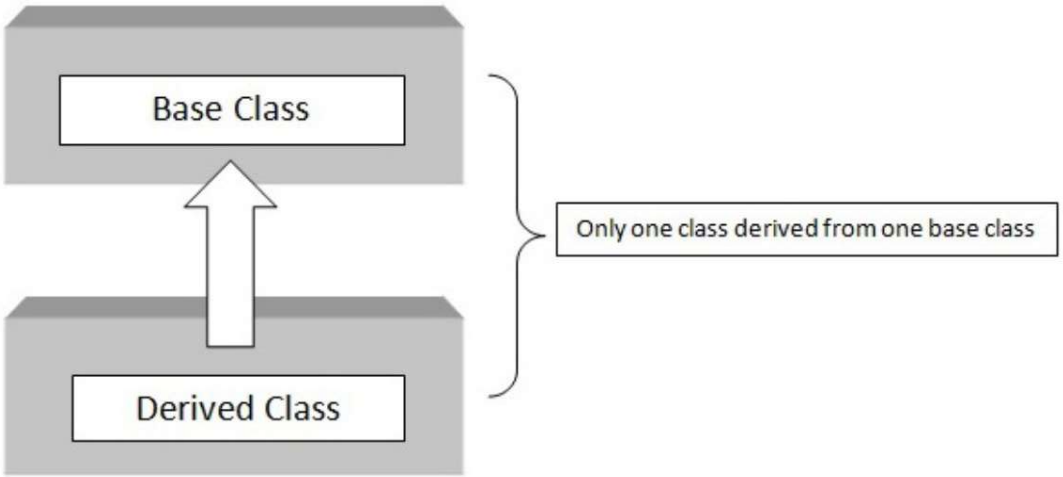
One parent → Many children

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

Types of Inheritance in Python

Single Inheritance

```
class Person:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Name:", self.name)


class Student(Person):
    def study(self):
        print(self.name, "is studying")


s = Student("Ram")
s.show()
s.study()
```



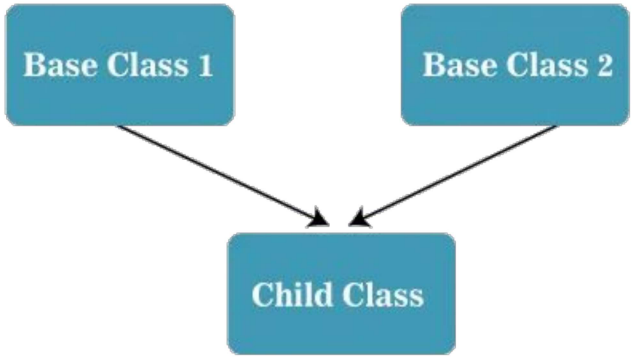

```
Name: Ram
Ram is studying
```

✔ Child gets all parent methods
✔ No duplication
✔ IS-A relationship (Student IS-A Person)

Types of Inheritance in Python

Multiple Inheritance

```
class Parent1:
    pass

class Parent2:
    pass

class Child(Parent1, Parent2):
    pass
```



Example:

```
class Father:
    def skills(self):
        print("Gardening skills")


class Mother:
    def talents(self):
        print("Painting talent")


class Child(Father, Mother):
    def hobbies(self):
        print("Reading books")


c = Child()
c.skills()    # from Father
c.talents()   # from Mother
c.hobbies()   # own method
```

```
Gardening skills
Painting talent
Reading books
```

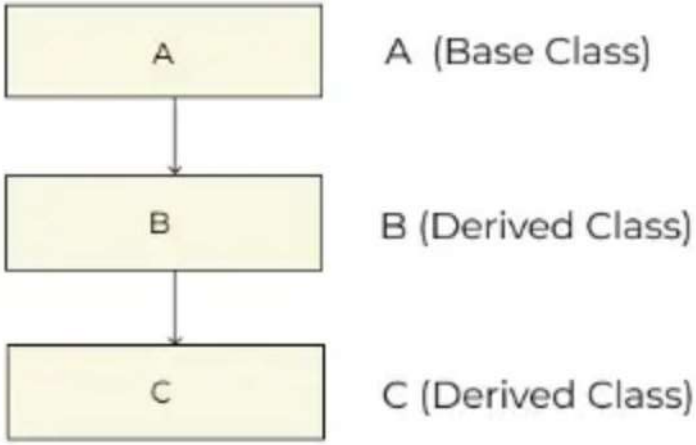
Types of Inheritance in Python

```
class Teacher:
    def teach(self):
        print("Teaching students")


class Researcher:
    def research(self):
        print("Doing research work")


class Professor(Teacher, Researcher):
    def guide(self):
        print("Guiding PhD students")


p = Professor()
p.teach()
p.research()
p.guide()
```

```
Teaching students
Doing research work
Guiding PhD students
```

Types of Inheritance in Python

Multilevel Inheritance




```
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```

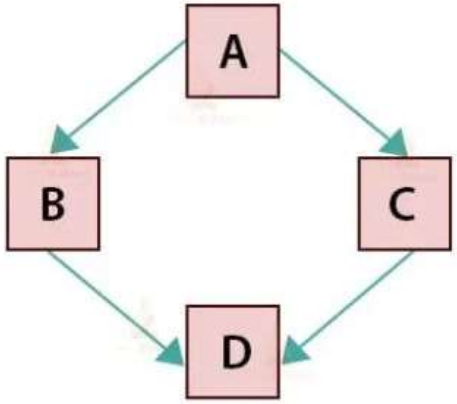
```
class Person:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print("Name:", self.name)

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def show_emp(self):
        print("Employee ID:", self.emp_id)

class Manager(Employee):
    def __init__(self, name, emp_id, department):
        super().__init__(name, emp_id)
        self.department = department

    def show_dept(self):
        print("Department:", self.department)
```

```
m = Manager("Ram", 101, "IT")
m.show_name()
m.show_emp()
m.show_dept()
```

```
Name: Ram
Employee ID: 101
Department: IT
```

Types of Inheritance in Python

Hybrid Inheritance



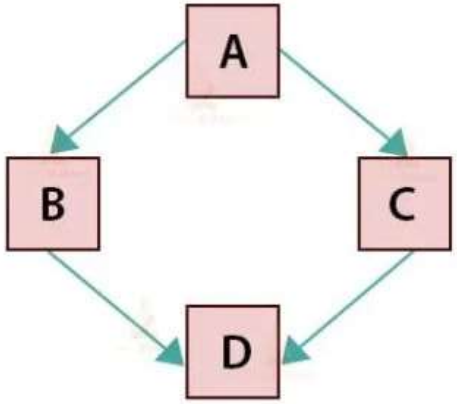
- Hierarchical (A → B, A → C)
- Multiple (D inherits B and C)
- Multilevel (A → B → D)

```
class A:
    def show(self):
        print("Class A")

class B(A):
    def show(self):
        super().show()
        print("Class B")

class C(A):
    def show(self):
        super().show()
        print("Class C")

class D(B, C):
    def show(self):
        super().show()
        print("Class D")

obj = D()
obj.show()
```

```
Class A
Class C
Class B
Class D
```

Types of Inheritance in Python

Hybrid Inheritance



- Hierarchical (A → B, A → C)
- Multiple (D inherits B and C)
- Multilevel (A → B → D)

```
class Person:
    def details(self):
        print("Basic personal details")

class Employee(Person):
    def work(self):
        print("Employee work")

class Student(Person):
    def study(self):
        print("Student studies")

class TeachingAssistant(Employee, Student):
    def role(self):
        print("Teaching Assistant role")
```

```
ta = TeachingAssistant()
ta.details()
ta.work()
ta.study()
ta.role()
```

```
Basic personal details
Employee work
Student studies
Teaching Assistant role
```

MCQ 1

```
class A:
    def __init__(self):
        print("A")

class B(A):
    def __init__(self):
        print("B")
        super().__init__()

class C(B):
    def __init__(self):
        super().__init__()
        print("C")

obj = C()
```

What is the output?

A) A B C
B) B A C
C) B C A
D) A C B

MCQ 2

```
class A:
    def __init__(self):
        print("A constructor")

class B(A):
    def __init__(self):
        print("B constructor")

obj = B()
```

What is the output?

A) A constructor
B) B constructor
C) A constructor B constructor
D) Error

MCQ 3

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        super().show()
        print("B")

obj = B()
obj.show()
```

Output?

A) A

B) B

C) A B

D) B A

Assignment 1 - SUBMIT BY 16-03-2026

1. Implement Bubble Sort and Merge Sort. Conduct experimental analysis on random and nearly sorted datasets. Evaluate their performance based on execution time, stability, and theoretical complexity. **[BT 5] [CO1]**

2. Implement Recursive Fibonacci and Iterative Fibonacci. Measure the number of function calls and execution time for increasing input sizes. Analyze why the recursive version exhibits exponential growth and suggest improvements. **[BT 4] [CO1]**

3. Implement insertion and deletion operations at different positions in an array. Analyze the time complexity of each operation and justify the limitations of arrays for dynamic data handling. **[BT 4] [CO2]**

4. Implement a Singly Linked List with insertion (beginning & end), deletion, and search operations. Compare its performance with array implementation and evaluate which structure is more efficient under different scenarios. **[BT 5] [CO2]**

5. Design an abstract class StackADT and implement two concrete subclasses: ArrayStack and LinkedStack. Experimentally evaluate their performance for large datasets and justify which implementation is more suitable for real-world applications. **[BT 5] [CO3]**

6.Design an abstract class StackADT and implement two concrete subclasses: ArrayStack and LinkedStack. Experimentally evaluate their performance for large datasets and justify which implementation is more suitable for real-world applications.**[BT 5] [CO3]**

What is an Abstract Class?

In Python, an abstract class:

- Cannot be instantiated directly
- May contain:
  - Abstract methods (must be implemented in child classes)
  - Concrete methods (already implemented and shared)
- Ensures **code reusability + enforcement of structure**

Python provides abstract class support using the **abc (Abstract Base Class)** module.

Basic Syntax:

```
from abc import ABC, abstractmethod

class ClassName(ABC):

    @abstractmethod

    def method_name(self):

        pass
```

Problem Without Abstract Class (Redundant Code)

Example: Payment System

Suppose we have multiple payment types:

- CreditCardPayment
- DebitCardPayment
- UPIPayment

Without Abstract Class

```
class CreditCardPayment:
    def validate(self):
        print("Validating payment...")

    def pay(self, amount):
        print("Processing credit card payment:", amount)


class DebitCardPayment:
    def validate(self):
        print("Validating payment...")

    def pay(self, amount):
        print("Processing debit card payment:", amount)
```

- validate() method is repeated
- No guarantee every class implements pay()
- Hard to maintain in large systems.
- If we change logic later then we must change in many places

Solution

```
from abc import ABC, abstractmethod
class Payment(ABC):
    def validate(self):
        print("Validating payment...")

    @abstractmethod
    def pay(self, amount):
        Pass
```

- Forces implementation
- Easier maintenance
- events misuse

Child Classes:

```
class CreditCardPayment(Payment):
    def pay(self, amount):
        print("Processing credit card payment:", amount)


class DebitCardPayment(Payment):
    def pay(self, amount):
        print("Processing debit card payment:", amount)
```

```
payment1 = CreditCardPayment()
payment1.validate()
payment1.pay(5000)

payment2 = DebitCardPayment()
payment2.validate()
payment2.pay(2000)
```

Stack ADT (Abstract Data Type)

A stack is a linear collection of items where insertion and removal happen only at one end called the "top."

- It follows the Last-In-First-Out (LIFO) principle, meaning the last item pushed is the first one popped.
- Imagine a stack of plates: you add (push) plates on top and remove (pop) plates from the top only.
- The main operations are:
  - push(x): Add element x to the top
  - pop(): Remove the top element
  - peek()/top(): View the top element without removing it
  - isEmpty(): Check if stack is empty

Stack Visual Representation Example:

Imagine the stack initially empty:

1. Stack: [ ]
2. Push elements 5, 10, 15:
3. Stack: [15 (top), 10, 5]
4. Pop one element removes 15:
5. Stack: [10 (top), 5]
6. Peek shows 10 without pop.

Applications of Stacks

1. Expression evaluation and syntax parsing
2. Browser history navigation (back button)

```python
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return len(self.items) == 0

    def push(self, item):
        self.items.append(item)
        print(f"Pushed {item}: {self.items}")

    def pop(self):
        if self.isEmpty():
            print("Stack is empty. Cannot pop.")
            return None
        popped = self.items.pop()
        print(f"Popped {popped}: {self.items}")
        return popped
```

```python
    def peek(self):
        if self.isEmpty():
            print("Stack is empty.")
            return None
        print(f"Top element: {self.items[-1]}")
        return self.items[-1]

stack = Stack()
stack.push(5)    # Stack: [5]
stack.push(10)   # Stack: [5, 10]
stack.push(15)   # Stack: [5, 10, 15]
stack.peek()     # Top element: 15
stack.pop()      # Removes 15, Stack: [5, 10]
stack.pop()      # Removes 10, Stack: [5]
stack.pop()      # Removes 5, Stack: []
stack.pop()      # Stack is empty, cannot pop
```

```
Pushed 5: [5]
Pushed 10: [5, 10]
Pushed 15: [5, 10, 15]
Top element: 15
Popped 15: [5, 10]
Popped 10: [5]
Popped 5: []
Stack is empty. Cannot pop.
```

What is a Queue?

A Queue is a linear data structure that follows the FIFO (First In, First Out) principle.

- First In , First Out
- The element inserted first is removed first.
- Think of it like a line of people waiting for a ticket:
  - The person who comes first gets the ticket first.

Key Terms

Front – The position of the first element in the queue (to be removed).

Rear – The position of the last element in the queue (most recently added).

Queue ADT

- A queue allows insertion at the rear and removal from the front, enforcing FIFO order across all accesses and updates.
- Typical queue states track front, rear, and size, with overflow/underflow considerations in fixed-capacity arrays and circular buffers.

Operation	Description
enqueue(x)	Insert element x at the **rear**.
dequeue()	Remove and return element from the **front**.
peek() / front()	Return element at **front** without removing.
isEmpty()	Check if the queue is empty.
isFull()	Check if the queue is full (for fixed-size queues).

```python
class Queue:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return len(self.items) == 0

    def enqueue(self, item):
        self.items.append(item)  # Add at rear

    def dequeue(self):
        if self.isEmpty():
            return None
        return self.items.pop(0)  # Remove from front

    def peek(self):
        if self.isEmpty():
            return None
        return self.items[0]
```

```python
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print("Front element:", q.peek())  # 10
print("Dequeue:", q.dequeue())     # 10
print("Dequeue:", q.dequeue())     # 20
print("Front element now:", q.peek())  # 30
```

```
Front element: 10
Dequeue: 10
Dequeue: 20
Front element now: 30
```

Applications

- Graph algorithms: breadth-first search (BFS) explores neighbors level-wise using a queue.
- Order processing and service systems: fair handling of requests by arrival time